



NewBear Computing Store

A Division of NEWBURY LABORATORIES LTD.



SALES & SERVICE: BONE LANE NEWBURY BERKSHIRE RG14 5SH Tel: 0635 46898 Telex: 888815

DESIGN NOTE 36

77-68 VDU BOARD

1. INTRODUCTION
2. BOARD CHARACTERISTICS
3. GENERAL DESCRIPTION
4. CIRCUIT DESCRIPTION
5. CONSTRUCTION
6. TESTING
7. DIAGNOSTICS
8. USER NOTES
9. 77-68 BUG 2

Figs;

1. SCHEMATIC
2. COMPONENT SIDE VIEW OF BOARD
3. RIBBON CABLE
4. COMPONENTS
5. MEMORY MAP
6. CONNECTOR WIRING
7. SIMPLE ASCII KEYBOARD

77-68 VDU-1 BOARD

1. INTRODUCTION

This Visual Display Unit board gives the 77-68 user fast and silent communication with his computer. It drives a standard television set or monitor giving an upper & lower case alphanumeric display of 24 lines of 40 characters each, and will accept input from a separate alphanumeric keyboard.

By designing the board as part of the 77-68 computer, it has been possible to treat the display refresh memory as a part of the main computer memory. Thus each character position on the screen corresponds to a CPU memory location, and can be individually written into and read from by the CPU. The screen contents can therefore be changed very rapidly, and also any individual character can be changed without affecting the rest of the display.

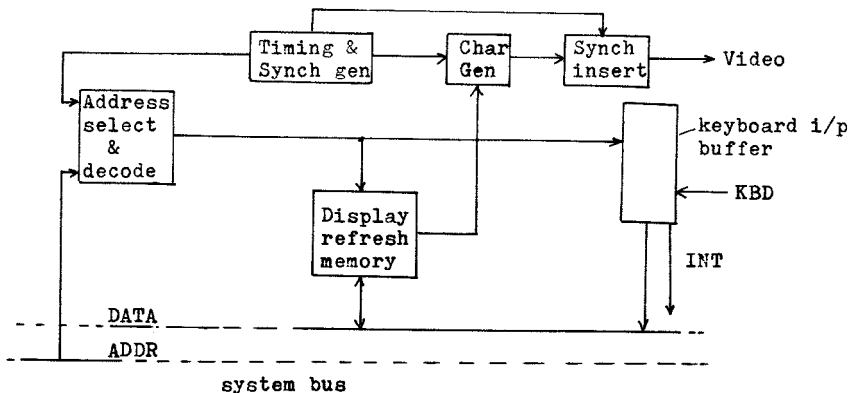
Individual characters may be displayed as white on a black background, or as black on a white background. This feature is useful for drawing attention to certain parts of the display, for differentiating between human input and computer output, or for giving a limited 'graphics' capability by treating the screen as a 24 x 40 matrix of black or white rectangles.

The printed circuit board available for the VDU also contains a small 'prototyping' area for those who may wish to try different character generators or add other circuitry.

2. BOARD CHARACTERISTICS

Construction	; 8.0" x 8.0" single sided PCB with gold plated edge connector contacts. Mechanically & electrically compatible with 77-68 CPU.
Bus	; Buffered. Compatible with 77-68 CPU bus.
Power	; Requires +5V stabilised DC @ 0.8A typical.
Video o/p	; Composite (video + synch) nominal +1V peak (white) when driving 75 ohm load. For direct connection to 625 line TV monitor or via UHF modulator to TV set.
Display	; 24 lines of 40 characters. Upper & lower case alphanumeric, black on white or white on black. Each character position defined by 7 x 10 dot matrix.
Keyboard i/p	; Parallel input port; 8 data lines plus strobe. 74LS compatible inputs. Positive pulse on strobe latches data and causes Non-Maskable Interrupt to CPU.
Addressing	; 960 bytes of read/write display refresh memory which may also be accessed by CPU, plus keyboard i/p buffer. Addresses in range F800 - FBFF (see Fig 5)

3. GENERAL DESCRIPTION



The display refresh memory consists of one eight bit byte for each of the 960 (24 x 40) display locations. The information contained in each location is a 7 bit character code - usually ASCII, but this will depend on the character generator chip used - while the eighth bit defines whether the character will be displayed as white on black or as black on a white background. The particular byte accessed at any instant depends on the state of the memory address inputs from the address select & decode circuit.

The Timing & Synch Gen circuits produce the horizontal and vertical synchronisation signals, tell the character generator which line of a character is to be displayed, and send an address to the refresh memory - via the address select circuits - defining which of the 960 character positions is to be displayed at that particular moment of the TV frame. Video blanking signals are also produced during the unused 'edges' of the TV frame.

The Character Generator circuitry accepts information defining the character to be displayed, whether it is to be displayed as white or black, and the particular line of the character being scanned. From this information it produces a video signal which is then mixed with the horizontal and vertical synchronisation signals to produce the composite video output.

During normal operation the Address Select circuits route the address generated by the Timing circuits to the display refresh memory so that each of the 960 bytes is accessed during a TV frame. However, the address decoding circuits are continually monitoring the system address bus looking for a CPU read or write operation which references the VDU board memory (i.e. with an address in the range F800 - FBFF). When this occurs the address selector circuits connect the lower 10 lines of the system address bus to the display refresh memory, in place of the address from the TV timing circuits. This change of address lasts for only about 1µs, and so is hardly noticeable on the TV screen, but allows the CPU to read or write one of the VDU refresh memory locations.

The Keyboard Input Buffer responds to a positive pulse on the Key Strobe input by latching the information present on the 8 keyboard input lines into an eight bit register. It then passes a Non-Maskable Interrupt to the CPU. The CPU will be programmed to respond and

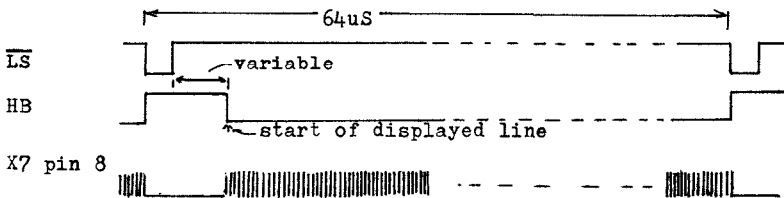
read out the information from the keyboard buffer (location FBFF). This address will be detected by the VDU board address decode circuits which will signal the keyboard input buffer to transfer the latched information onto the system data bus and clear the Interrupt signal.

4. CIRCUIT DESCRIPTION

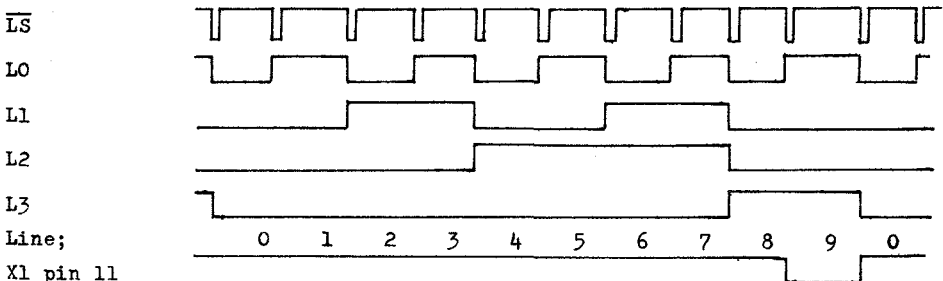
The display timing and TV synchronisation signals are derived from a free running oscillator comprising R9,R10,RV3,C2,C3 and half of the dual timer X15. This oscillator produces short pulses at the TV line scan rate (approx 15kHz) at its output (X15 pin 5).

These pulses are used as the Line Synchronisation signal ($\overline{LS}$) and also drive the second half of X15 which is connected as a variable pulse width monostable. The output of this second half of X15 (HB) is used to blank the video signal during the line synchronisation and flyback periods and for a variable time afterwards - allowing the user to adjust the position of the start of the displayed character rows by varying RV2.

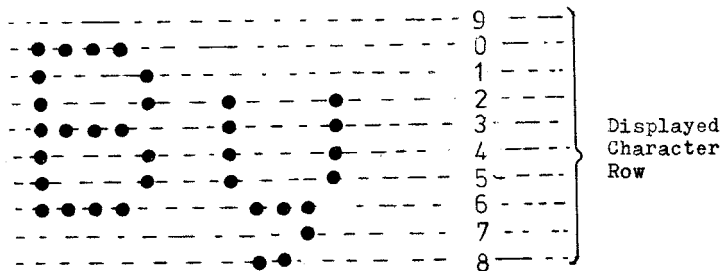
During this blanking period (HB = 1), X12 pin 11 goes low, which holds X7 pin 6 high. However, at the end of the HB pulse, X12 pin 11 goes high, allowing X7 (with R6,RV1,C1) to operate as a free running 'dot' oscillator at about 7MHz. It is this oscillator which controls how rapidly the characters in a display line are generated, and thus the width of the displayed row of characters. The second half of X7 is used as a buffer to ensure a good square waveform at X7 pin 8;



Each of the 24 character rows of the display is made up of 10 TV scan lines, hence by dividing the $\overline{LS}$ signal by 10 in X11, signals L0-L3 are generated to define which of the lines within a character row is being scanned;

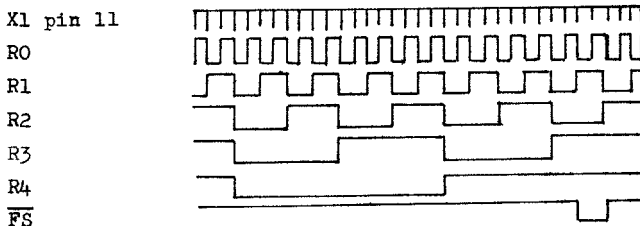


The particular character generator used (74S262 , X26) uses lines 0 to 6 inclusive to display most characters (lines 7 & 8 are used for the descenders of lower case characters such as 'j'), but a better looking display results if the characters are more central within the boundaries of the individual display 'cells'. To do this means that the top line of the character should be the second line of the 'cell';



To achieve this, the counter which defines the character row is designed to change state at the beginning of the line before line 0.

To do this, X1 is arranged to generate a '0' going edge on its output pin 11 at the start of line 9. This edge triggers the divide by two X10, and the output of X10 is further divided by sixteen in X9, giving the character row defining signals R0 - R4;



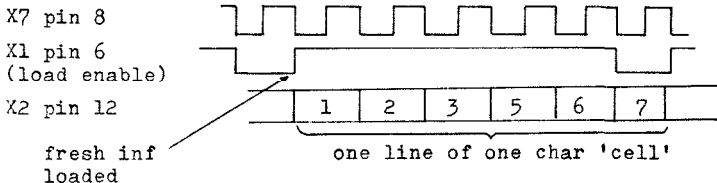
Parts of X16 & X8 give a '0' pulse ($\overline{FS}$) during character row number 26. This is used as the TV Frame Synchronisation signal.

This arrangement of Line and Row counters gives a non-interlaced scan of $10 \times 32 = 320$ lines which, although different from the $312\frac{1}{2}$ lines in a normal TV half-frame, is nevertheless close enough to be acceptable by virtually any '625 line' TV set.

As mentioned previously, X7 produces approximately 7MHz while HB is low. This 7MHz signal is fed to X6 which is connected as a synchronous divide by 7, there being 7 'dot' positions in each display cell. X6 is reset by HB at the start of each line.

X1 pin 6 output consists of '0' pulses at $1/7$ th of the 'dot' oscillator frequency. These are fed to the synchronous divide by 64 counter X14,13 whose outputs C0 - C5 define which display column is valid at any time. Again, X13 & X14 are reset to zero at the start of each line by HB.

The '0' pulses at X1 pin 6 also cause the four bit shift registers X4,X2 to load the information present at their parallel inputs (ABCD). As these shift registers are clocked by the 7MHz signal from X7, the seven bits of information loaded from X3 & X5 will be shifted out via X2 pin 12 before X2 & X4 are re-loaded. These seven bits comprise the seven 'dots' of a line of a display cell;



By connecting the 'J' input of X4 to OV, the information that is loaded into the first stage of X4 (from the 'A' input) is kept in this first stage, and is not shifted out by the 7MHz serial clock. Thus any information presented to the A input of X4 during a 'Load Enable' pulse (X1 pin 6 at '0') will be transferred to the Qa output and remain there until the next Load Enable pulse.

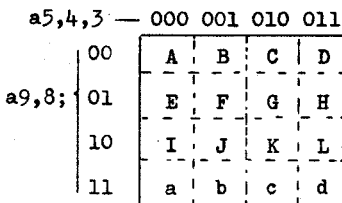
This facility is used for blanking the video output. If any of the open collector outputs of X17 or X12 go low, then X1 pin 1 goes low for the corresponding character output period and inhibits the transmission of the video signal. Since the video signal at pin 3 of X1 is inverted, another part of X1 is used to re-invert it to give the correct polarity at X1 pin 8.

D1,D2,R3 & R4 mix the video and synchronisation signals in the correct amplitude ratios, then the composite signal is buffered by the emitter follower Q1. R1,R2 reduce the signal amplitude to nominal 1V peak (into a 75 ohm load) and present a reasonable approximation to 75 ohm impedance at the output.

The actual character generator chip is X26. In response to the 7 bit character code applied to inputs Ca - Cg, and a four bit input (L0-3) defining the line being scanned, X26 produces five outputs (Y1-5) corresponding to the five possible character 'dot' positions. These, together with two more 'dots' corresponding to the 'borders' of the character cell, are fed to the shift register X4, X2 via exclusive or gates X3,5. These either pass the signals unaltered, or invert them, according to the state of the eighth data bit (D07).

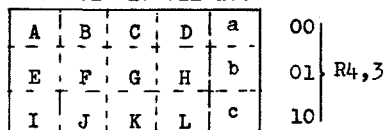
X27-34 form a 1024 x 8 bit memory, the data outputs of which may be fed to the system data bus via the tri-state buffer X25. In practice only 960 of the 1024 locations are used.

A problem arises due to the fact that there are 10 address inputs to the memory IC's X27-34, but the TV timing circuits naturally define the displayed character position by 11 lines (R0-4 & C0-5). This arises because although X27-34 can be considered to be a 32 x 32 array of bytes, the TV display is in a 40 x 24 format. To overcome this mis-match, 8x8 byte 'blocks' of memory are 'moved' as;



32 x 32
(4x4 blocks of 8x8)

c5,4,3 — 000 001 010 011 100



('d' addresses used for keyboard buffer)

40 x 24
(5x3 blocks of 8x8)

The following rules accomplish this 'shift' by generating a 10 bit memory address (a0 - a9) from the 11 line timing signals (R0-R4,C0-C5);

a0,1,2 = C0,1,2

a5,6,7 = R0,1,2

If C5 = 0 then a3,4 = C3,4 , a8,9 = R3,4

If C5 = 1 then a3,4 = R3,4 , a8,9 = 1,1

Thus X22 selects either C3 & C4 or R3 & R4 according to the state of its 'A' input (= C5), and X21 outputs are either equal to R3 & R4 (C5 = 0) or '1' (C5 = 1).

If the system bus address lines A11-A15 are all at '1', and A10 is at '0', then the 1/8 decoder X19 will be enabled (G1=1,G2,3=0). These conditions correspond to addresses in the range F800-FBFF.

When enabled, X19 produces a low signal on one of its outputs according to the state of its A,B & C inputs. If X18 pin 8 is low (keyboard buffer address), and both E (Enable) and R/W (Read/Write) inputs are high, then the KBD line will go low to enable the tri-state output buffers in X24.

However, if X18 pin 8 is high when X19 is enabled (VDU memory addressed), then one of the high four X19 o/p's 4,5,6 or 7 will go to 0;

R/W	E	X19 o/p			
		4	5	6	7
0	0	0	1	1	1
0	1	1	0	1	1
1	0	1	1	0	1
1	1	1	1	1	0

(Write from CPU to mem)

(Read from mem to CPU)

By feeding all four of these latter outputs to the NOR gate X18, a signal is generated on X18 pin 6 which goes to '1' whenever the system address bus references the VDU memory. This signal controls the address multiplexing circuits X20,22,23 to take the refresh memory address from the system data bus when X18 pin 6 is high, from R0-4, C0-5 when it is low.

When X18 pin 6 goes high, C4 charges rapidly positive via D3, making X12 pin 6 go low and hence blanking the video output. On X18 pin 6 going low again (when the CPU has finished accessing the VDU memory) C4 discharges through R11, keeping the video output blanked until X27-34 have had time to react to the revised address.

5. CONSTRUCTION

Refer to Figs 2,3,4

The close track spacing necessary, particularly around X24 - 34, means that great care has to be taken to avoid short circuits. The constructor should take care to make good soldered joints using the minimum amount of solder while ensuring that a joint is made all round the component pin or wire end. Use of a small soldering iron fitted with a small, clean, bit is essential.

The use (or not) of IC sockets is largely a matter for personal preference. However, poor quality sockets must be avoided as they can cause many hard to trace faults. Provided that the constructor is confident of his ability to solder them in the right way round first time, and is using a low leakage soldering iron, there is no reason why the IC's should not be soldered directly into the board. In case of trouble, remove a suspect IC by first cutting its body free from all its leads, then remove the remaining lead stubs one at a time. This procedure ruins the IC but does least damage to the board.

The best order of construction is;

Make sure that RV1,2 & 3 will fit, enlarging the board holes if necessary, but do not solder them in at this stage.

Examine the track side of the board carefully for any swarf left over from the drilling process, for unetched copper 'bridges' between tracks, and for possible small breaks in the tracks.

Fit all the straps (not the ribbon cable at this stage) using sleeved wire where appropriate.

Fit all IC's (and sockets if used) except for X27-34.

Fit the ribbon cable as shown in Figs 2 & 3.

Fit the remaining components (except X27-34).

Clean the track side of the board thoroughly and examine it for potential short circuits caused by bent leads, excess solder on joints, solder splashes etc.

Finally, test with an ohm meter for possible short circuits between adjacent tracks in the X24-34 area. Some reading is to be expected due to IC internal resistances, but a true short circuit is a fault.

Note; X27-34 will be fitted after preliminary tests, when the board should again be cleaned and checked for short circuits.

6. TESTING

- a) Before fitting X27-34, and before fitting the board into the system, the timing and TV synchronisation signals may be checked by connecting the board to the display and to a 5VDC supply.

By adjusting RV1,2 &3, a stable display should be obtained of a white 40 x 24 grid on a black background.

- b) Fit X27-34 and check again for short-circuits between the tracks connecting these IC's.
- c) Check the wiring to the VDU board socket, then plug the board into the system and switch on while monitoring the +5V supply. Leave switched on for a few minutes while checking all components on the VDU board for signs of overheating. (X24,26 and to some extent X27-34 will get warm, but no component should get so hot as to be painful to touch).
- d) With the system Reset button held operated, a stable display of random characters should be obtained.
- e) Check that the rest of the system is still working by loading and running a previously proven program.
- f) Load and run the following program;

```
FF00 B1 F0FF START CMP A SWREG    is switch reg setting
      03 27 FB      BEQ  START      same as before ?
      05 B6 F0FF    LDA A SWREG    if not get new setting
      08 CE F800    LDX  #F800     and fill display with
      0B A7 00     LOOP STA A 0,X   that character
      0D 8C FBFF    CPX  #FBFF
      10 27 EE      BEQ  START
      12 08        INX
      13 20 F6     BRA   LOOP
```

This program monitors the control panel switch (data) register, and whenever a switch setting is changed, fills the display with the character corresponding to the new code set up. By running through the various switch combinations, the character generator and the video inversion circuitry may be checked.

- g) The following program loads successive display locations with different characters, and checks that the CPU can read them back from the VDU refresh memory. If all is well the display will continually shift, but if an error occurs (in that the information read out from the refresh memory is not what was expected), the display will halt. Note that, for simplicity, the program only scans the first 32 display columns.

```
FF00 CE F800 START LDX  #F800
      03 A7 00     LOOP1 STA A 0,X
      05 A1 00     CMP  A 0,X
      07 26 0C     BNE  ERROR
      09 4C        INC  A
      0A 5C        LOOP2 INC B      delay to make shifting
      0B 26 FD     BNE  LOOP2      pattern more visible
      0D 08        INX
      0E 8C FB00    CPX  #FB00
      11 26 F0     BNE  LOOP1
      13 20 EB     BRA   START
      15 3E        ERROR WAI      crash here
```

h) To test the keyboard input buffer, load and run the following;

```
FF00 8E F080 START LDS #F080 set up stack
03 20 FE LOOP BRA LOOP hang about
05 B6 FBFF INTV LDA A KBUFF get character from key buffer
03 B7 F0FF STA A DISPL display it on panel lamps
0B 3B RTI
```

FFFC FF 05

NMI vector

This program normally runs in an endless loop (LOOP BRA LOOP), but entering a character via the keyboard will interrupt the CPU and make it jump to the INTV routine. This gets the character from the buffer and displays it on the panel lamps before returning to the endless loop.

7. DIAGNOSTICS

To be performed with the board removed from the system and supplied with +5V.

Test signals to be applied to board connector;

'0' = direct connection to 0V

'1' = connection to +5V via 1k ohm resistor

Measurements made with 20k ohm/Volt or better meter;

'0' = 0 to 0.4V

anything inbetween is wrong unless otherwise stated

'1' = +2.4 to +5V

a) Line oscillator & horizontal position delay (X15)

Best checked with an oscilloscope (see section 4 for waveforms), but if one is not available then X15 pin 1 should be about +2.5V when checked with a moving coil meter, and X15 pin 12 should be about 0.6V ; and should vary slightly as RV2 is adjusted.

b) Dot oscillator (X7)

A high frequency oscilloscope is needed to check this properly, otherwise X7 pins 6 & 8 should measure about +2V.

c) Divide by 7 (X6)

When checking with an oscilloscope, triggering is easier if the dot oscillator is made to run continuously by temporarily connecting HB (X12 pin 12) to 0V. Without an oscilloscope the operation of this circuit can be checked roughly by measuring the voltage on X1 pin 6 (should be about +2V) and noting that it changes (either rises slightly or falls almost to 0V) when X7 pin 5 is temporarily connected to 0V.

d) Character column counter (X14,13)

If checking with an oscilloscope (see waveforms in section 4), RV1 & RV2 must be adjusted correctly to set up the right conditions for a complete count (i.e. for C5 (X13 pin 13) to change state). If only a moving coil meter is available, check that points C1 - C4 are all at about +2V.

e) Character generator X26

For a full check run the program in section 6, but a limited test can be done as follows;

- temporarily connect $\overline{MW}$ (X19 pin 10) to OV. If the memory (X27-34) and the data output buffer (X25) are OK this should result in pin 12 of each of X27-34 being at '1'.
- then check with a moving coil meter that the Y1-Y5 outputs of X26 are at approximately +1.5V (1:1 square wave on 'scope).

f) Video inverter (X3,5)

With the memory set to give '1's as (e), check that;

X5 pins 3,6 and X3 pin 8 are '1'

X5 pins 8,11 and X3 pins 3,6,11 are approx 1.5V (1:1 square wave).

g) Shift register (X2,4)

With memory set to give '1's as (e) and the dot oscillator made to run continuously by temporarily connecting HB to OV, check;

X4 pin 15 is '0' (if it isn't then check that the corresponding input - X4 pin 4 - is at '0' before condemning X4).

X4 pin 12 is approx 0.7V on a moving coil meter, 1:6 ratio '1':'0' on 'scope.

X2 pin 12 is approx +1V on moving coil meter, 2:4 ratio '1':'0' on 'scope.

h) Line counters X11,10,9 & $\overline{FS}$ generator X8,16 (parts)

If a 'scope is available check for the waveforms as given in section 3, otherwise - measuring with a moving coil meter -;

L0,L1,L2,R0-R4	approx +1.3V
L3	" 0.7V
X1 pin 11	" 2.5V
X8 pin 4	" 0.8V
X16 pin 6	" 2.5V when X1 pin 8 con to OV

i) Address decoding (X16,18, parts)

Apply '1's to A3,4,8,9 inputs. Check that X18 pin 8 is '0'.

Change A3,4,8,9 in turn to '0'; in each case X18 pin 8 should be '1'.

Apply '1's to A12-A15 inputs. Check that X16 pin 8 is '0'.

Change A12-15 inputs in turn to '0', in each case X16 pin 8 should be '1'.

j) Address, E,R/W decoding (X19, X18(part))

With A3,4,8,9,12-15 inputs all set to '1'; check that;

- with E & R/W inputs at '0', X19 pin 11 goes to '0' only when A10 input is set to '0' and A11 to '1'.

- with A10 at '0' and A11 at '1', check X19 output pins 7-11 for different combinations of E and R/W inputs;

E	R/W	X19 pin;	11	10	9	7
0	0		0	1	1	1
1	0		1	0	1	1
0	1		1	1	0	1
1	1		1	1	1	0

also check that for each combination X18 pin 6 is '1'

- with E,R/W and A11 at '1', A10 at '0', check that X19 pin 12 is '1'. Change A9 input to 0 & check that X19 pin 12 goes to

'0', and X18 pin 6 goes to '0'.

k) Address selectors (X20,22,23)

With '0' applied at All input, X18 pin 6 should be '0', and the outputs (Y1, Y2 etc) of X20,22,23 should come from the TV timing R & C signals. This can really only be checked with a 'scope, but as a guide, the 10 RAM address lines should measure in the range 0.8 to 2.2V on a moving coil meter.

With A9,10 set to '0', and '1' applied to All-15, X18 pin 6 should be '1'. Under these conditions the Y1,Y2 etc. outputs of X20,22,23 should equal the appropriate address bus inputs A0-A9 (e.g. X20 pin 12 should be the same as X20 pin 13), and this may be checked by applying different signals to the address bus board inputs and checking the corresponding selector outputs.

l) Data output buffer (X25)

Apply '0' to the board R/W input; X25 pins 1 & 19 and X24 pin 1 should be '1'. Check that each data bus line (D0-D7) is in the high impedance state as follows;

- Temporarily connect a 1k resistor from the data bus line to +5V. The voltage across the resistor should be less than 0.2V.
- Temporarily re-connect the 1k resistor so that it is between the bus line and 0V. Measure the voltage across the resistor, it should be less than 0.2V.

Note; these checks also test the high impedance output state of the keyboard input buffer X24.

Set A9,10 inputs to '0', All-15,E & R/W to '1'. X25 pins 1,19 should then be '0', and X18 pin 6 should be '1'. These conditions enable the data buffer X25 and select the board address bus inputs for routing to the refresh RAM. Check that the output of each of the 8 buffers of X25 is at the same logic state as its input (i.e. X25 pins 3 and 2 should be the same). Since the inputs to X25 are coming from the (random) pattern stored in the refresh RAM, then by varying the inputs applied to the address inputs A0-8, it should be possible to find a memory cell containing a '1' and also one containing a '0'.

m) Keyboard input buffer (X24)

Apply '0' to A10, '1' to A3,4,11-15,E and R/W. Check that X24 pin 1 is '0'. Then;

- Apply '0' to 'KS' input. Momentarily apply '0' to RESET input. Check that NMI output is '1'.
- Apply '1' momentarily to 'KS' input. NMI output should go to '0'.
- Apply '1' to 'KS' input and keep it applied while '1' and '0' in turn to each of the K0-K7 inputs, checking each time that the corresponding data bus line (D0-7) is in the same logic state as the K input.

General Notes on Fault Finding

- Failure rates of IC's obtained from reputable suppliers are very low. If the board doesn't work then this is most probably due to poor workmanship e.g. dry joints, solder splashes, hair-line cracks in PCB tracks.
- Normal (non tri-state) IC outputs are never exactly 0.0V in the '0' state. If you do measure 0.0V on a gate output that is feeding inputs of other IC's, then suspect a short to 0V.
- With a little thought it is possible to deduce the likely cause

of most faults from an examination of the circuit diagram.

- Don't forget that X9,10,11 & X27-34 have their OV & +5V pins in unusual positions;

	<u>+5V pin</u>	<u>OV pin</u>
X9,11	5	10
X10	4	11
X27-34	10	9

All other IC's have OV at the 'bottom left'(pins 7,8,10 or 12), and +5V on the highest numbered pin.

8.USER NOTES

Most keyboards provide 7 data lines. Connect these to KO-6, KO being the least significant, and connect K7 input to OV (BUG 2 monitor requires the most significant bit of a character from the keyboard to be '0').

KS input should normally be '0', with a short (about 2uS) '1' pulse when new valid data is present on the KO-7 lines.

Because of limitations in the address decoding circuit, the keyboard input buffer is in fact accessed by any of the spare (1024 - 960) addresses (those in the range F800 to FBFF which do not correspond to any of the display cell addresses - see Fig 5). This would not matter except that a quirk in the 8212 IC (X24) makes it generate an interrupt whenever it is accessed by the CPU. The user should therefore beware of 'accidentally' programming a read from any of these 'spare' addresses.

If the keyboard input buffer is not used, then the user may gain access to all 1024 RAM locations by breaking the connection from X18 pin 8 to X19 pin 3.

If the VDU board is fed to a normal TV set via a modulator, the composite video output amplitude and the ratio of the video and synch components of this signal may need trimming to suit the characteristics of the particular modulator. Try adjusting R2 and /or R4

Because of memory space restrictions, a full keyboard interrupt (NMI) handler has not been implemented in BUG 2. The chief limitation is in the section of code;

```
        GETCH    WAI
                LDA A KBUFF2
```

as an IRQ interrupt (from another peripheral) could cause the program to pass to the LDA A KBUFF2 instruction erroneously. To implement a full interrupt handler, then;

- The NMI handler should not only save the character entered from the keyboard in a buffer area of memory, but it should also set a flag in memory to indicate that a new character has been accepted.
- The main program should periodically examine this flag, and if it finds it set it should take the character from the memory buffer area, and clear the flag.

To use the full 40 x 24 character display, a routine is required which will accept as input two bytes which define the desired character row (0-23) and column (0-39) and from them calculate the corresponding memory address.

9. 77-68 BUG 2

Purpose

77-68 BUG 2 is a minimal system monitor for use with the VDU1 board and an external parallel input ASCII keyboard. It is designed to be entered into the top 256 words of memory via the system control panel switches and may then be dumped onto paper tape or cassette tape for subsequent re-loading by the MON 1 board Bootstrap loader.

77-68 BUG 2 allows the user to enter and run programs using the keyboard/VDU, and to examine and alter the contents of registers and memory locations.

Because the monitor program has been limited to only 256 bytes, in order to allow it to be initially loaded via the control panel switches, its facilities are a bit restricted. Notably, there is no provision for saving and loading programs on an external media such as paper or magnetic tape, and the useable VDU display is limited to 16 lines of 32 characters. It is expected that BUG 2 will mainly be used in the development of more sophisticated monitor programs, tailored to the individual user's requirements and peripherals.

Use

When BUG 2 is loaded, operation of the system Reset button will start it running, clear the screen, and display a '*' at the beginning of a line.

When a user program is running, control may be transferred back to BUG 2 by;

- a SWI (Software Interrupt) instruction in the user program.
- or - typing CTRL/C (hold the CTRL key down while typing C).

In either case the display will scroll up one line and display a '*' at the start of the next line, and the 6800 MPU registers will have been saved on the stack. The saved registers may then be displayed (R), altered (A), then the user program continued (C) or started at a different location.

The Reset button should only be operated during the course of a user program if all else fails, as it is liable to destroy the MPU registers.

When the monitor displays '*', it will then wait (Run lamp out) for the user to enter one of the following single letter commands;

- A (Alter memory location)
- E (Examine memory locations)
- G (Go to start of user program & run it)
- R (Display contents of MPU registers resulting from run of user program)
- C (Continue to run user program after SWI or CTRL/C)

RAM

7768 BUG 2 uses RAM locations FOEF to FOFE for temporary storage, and also has its own stack extending below FOEE. The user program may either use the BUG 2 stack, or set up its own stack in a separate area of RAM.

E (Examine) Command

When BUG 2 is waiting for a command ('*' printed), type 'E', followed by the four digit hex address of the first memory location to be examined. The monitor will separate the E and the address by a space, and display the contents of 8 consecutive memory locations starting at the specified address. e.g.;

*E FFO0 B6 FB FF 81 03 27 14 B7 (user input underlined)

* (monitor ready for next command)

A (Alter) Command

Enter 'A', followed by a four digit hex address, then the two digit byte to be stored at that address;

*A 0000 12 (spaces displayed by monitor)

The user may then either;

- enter a non-hex character (e.g. space) to terminate the command;

*A 0000 12

*E 0000 12

or - enter another two hex digits to be entered in the next memory location;

*A 0000 1122334455667788

*E 0000 11 22 33 44 55 66 77 88

*

G (Go) Command

When the user has loaded a program (with the E & A commands), then to start it running enter G followed by a four hex digit program start address. The program will then be started and will run until it encounters a SWI instruction or CTRL/C is entered on the keyboard or the Reset button is operated.

C (Continue) Command

When a user program has been halted by a SWI or CTRL/C, and control has been returned to the monitor, then entering 'C' will cause the monitor to retrieve the old values of the MPU registers from the stack, and re-enter the user program at the point it left off.

R (Register Display) Command

Typing 'R' displays the contents of the MPU registers as they were saved on the stack by the last SWI or CTRL/C;

*R FO 1E 55 BE00 0010 0079

* CC B A X P S

P is the address of the location in the user program immediately following the point at which it stopped running and control was transferred to the monitor; it is therefore the address at which the program will continue running from when the C command is entered.

S is the user stack pointer content on entry to the monitor, after saving the MPU registers, and therefore in the above example the values of CC,B,A,X & P are stacked immediately above location 0079, thus;

*E 007A FO 1E 55 BE 00 00 10 ...

If altered;

*A 007C FF

*R FO 1E FF BE00 0010 0079

Then the altered value will be loaded into the MPU when the next 'C' or 'G' command is given.

Example of BUG 2 use

To load & run the following program;

<u>Loc</u>	<u>Instr</u>			
0000 4F	START	CLR A		;clear A,B & CC
01 06		TAP		
02 5F		CLR B		
03 8E 0080		LDS 0080		;define user stack
06 30		TSX		; & X reg
07 86 AA	LOOP	LDA A AA		; display '1010 1010' on panel
09 B7 F0FF		STA A DISPLAY		; lamps
0C 3F		SWI		; return to monitor
0D 86 55		LDA A 55		; display '0101 0101'
0F B7 F0FF		STA A DISPLAY		
12 3F		SWI		
13 20 F2		BRA LOOP		

*A 0000 4F065F8E00803086AA

*A 0009 B7F0FF3F8655B7F0FF3E20F2

*E 0000 4F 06 5F 8E 00 80 30 86

*E 0008 AA B7 FO FF 3F 86 55 B7

*E 0010 FO FF 3F 20 F2

*G 0000 panel lamps should change to '1010 1010'

*R C8 00 AA 0080 000D 0079

*C panel lamps should change to '0101 0101'

*R C0 00 55 0080 0013 0079

*C panel lamps should change to '1010 1010'

etc.

Useful BUG 2 routines

Various BUG 2 routines may be accessed (by a JSR instruction) by a user program. The most useful are;

GETCH (access with JSR FF7B); turns Run lamp off until a character is entered on the keyboard, then displays it in the next screen position and returns with character in location KBUFF2 (FOFE).

PNTCH displays character in A. Access with JSR FF7F.

Interrupts

Because of space limitations, a full interrupt handler has not been implemented, Therefore IRQ interrupts should not be used in a program to be run with this version of the monitor.

77-68 BUG 2

256 byte Monitor for 77-68 with VDU-1 board and parallel keyboard input.

TOPSTK	EQU	\$FOEE	top of BUG stack
TMPXHI	EQU	\$FOF6	temp storage
TMPXLO	EQU	\$FOF7	" "
TMPX2	EQU	\$FOF8	" "
TMPSTK	EQU	\$FOFA	" "
VDUADR	EQU	\$FOFC	addr of next display pos'n
KBUFF2	EQU	\$FOFE	char i/p buffer
KBUFF	EQU	\$FBFF	K/B i/p buffer

Non Maskable Interrupt handler. Takes character from keyboard buffer, jumps to Monitor if it was CTRL/C, otherwise stores character in KBUFF2 & returns.

```
FF00 B6 FBFF NMIHDL LDA A KBUFF    get char
      03 81 03      CMP A #03      CTRL/C ?
      05 27 14      BEQ SWINT
      07 B7 FOFE     STA A KBUFF2   save char
      0A 3B         RTI             & return
```

Reset handler, sets up BUG stack pointer & clears screen.

```
FF0B 8E FOEE RESET  LDS #TOPSTK
      0E CE F800     LDX #F800
      11 86 20      LDA A #20      (= space)
      13 A7 00      CLRALL STA A 0,X
      15 08         INX
      16 8C FCOO     CPX #FC00     end of display ?
      19 26 F8      BNE CLRALL
```

SWI handler, saves user program stack pointer.

```
FF1B BF FOFA SWINT  STS TMPSTK
```

RSTART; Monitor returns here after executing each command. Switches to BUG's own stack, displays '*', & gets new command character.

```
FF1E 8E FOEE RSTART LDS #TOPSTK
      21 8D 6E      BSR SCROLL    new line
      23 86 2A      PNT*  LDA A #'*
      25 8D 58      BSR PNTCH     print *
      27 8D 52      BSR GETCH     get command
```

Sort out which command was entered & go to routine. To RSTART if command not valid.

```
FF29 81 41         CMP A #'A      Alter ?
      2B 27 2F      BEQ ALTER
      2D 81 43      CMP A #'C      Continue ?
      2F 27 27      BEQ CONT
      31 81 47      CMP A #'G      Go ?
      33 27 14      BEQ GOGOGO
      35 81 52      CMP A #'R      Reg print ?
      37 27 2C      BEQ REGPNT
      39 81 45      CMP A #'E      Examine ?
      3B 26 E1      BNE RSTART
```

'Examine' routine; get address, print contents of that & 7 following locations.

```

FF3D 8D 72      BSR  GETADR
    3F C6 08      LDA  B #88
    41 8D 68      EXAMLP BSR  P2H&S      display contents
    43 08          INX                      point @ next
    44 5A          DEC  B
    45 26 FA      BNE  EXAMLP
    47 20 DA      BRA  PNT*
```

Go to user program; get starting address & put it on user stack so that RTI will load it into prog. counter, CONT does the rest; restores user stack pointer then returns to user program.

```

FF49 8D 66      GOGOGO BSR  GETADR
    4B FE FOFA      LDX  TMPSTK      point @ user
                                   stack.
    4E B6 FOF6      LDA  A TMPHXI      change PC on
    51 A7 06      STA  A 6,X          user stack
    53 B6 FOF7      LDA  A TMPXLO
    56 A7 07      STA  A 7,X
    58 BE FOFA CONT LDS  TMPSTK      restore user SP
    5B 3B          RTI                over & out
```

'Alter' routine; get address of first location to be changed, then get data for that & subsequent locations. Routine HEX2IN jumps to RSTART when a non-hex character is entered on the keyboard.

```

FF5C 8D 53      ALTER BSR  GETADR      get address
    5E 8D 62      ALTLP BSR  HEX2IN      get data
    60 A7 00      STA  A 0,X          store it
    62 08          INX                point @ next loc.
    63 20 F9      BRA  ALTLP          back for more
```

'Register print' routine; point @ user stack & print registers from it, then print user SP.

```

FF65 8D 46      REGPNT BSR  PNTSP
    67 FE FOFA      LDS  TMPSTK      point @ stack
    6A 8D 3E      BSR  IP2H&S      display CC reg
    6C 8D 3C      BSR  IP2H&S      "    B    "
    6E 8D 3A      BSR  IP2H&S      "    A    "
    70 8D 35      BSR  IP4H&S      "    X    "
    72 8D 33      BSR  IP4H&S      "    PC   "
    74 CE FOFA      LDX  #TMPSTK
    77 8D 2F      BSR  P4H&S      disp user SP
    79 20 A3      START2 BRA  RSTART
```

Get character from keyboard; wait for interrupt caused by keyboard then load character into A.

```

FF7B 3E          GETCH  WAI
    7C B6 FOFE      LDA  A KBUFF2
```

Display character in A. Scroll if it is a Car Ret or if end of (32 char) line.

```

FF7F FF FOF8 PNTCH STX  TMPX2      save X
    82 FE FOFC      LDX  VDUADR      point @ display
                                   position
    85 81 0D          CMP  A #80D      Car Ret ?
    87 27 08          BEQ  SCROLL
    89 A7 00          STA  A 0,X      display char
    8B 08          INX                point @ next pos
```

FF8C 8C FA00	CPX	#FA00	past end of line?
8F 26 0F	BNE	ENDPNR	
91 CE F7E0	SCROLL LDX	#F7E0	
94 36	PSH A		
95 A6 40	SCRLUP LDA A	40,X	
97 A7 20	STA A	20,X	
99 08	INX		
9A 8C F9E0	CPX	#F9E0	end scroll with
9D 26 F6	BNE	SCRLUP	X = F9E0
9F 32	PUL A		
AO FF FOF8	ENDPNT STX	VDUADR	update VDUADR
A3 FE FOF8	LDX	TMPX2	restore X
A6 39	RTS		

Hex print routine; print one or two bytes pointed to by X as two or four hex digits plus space. X may be incremented before each byte is read.

FFA7 08	IP4H&S INX		
A8 8D 23	P4H&S BSR	P2HEX	
AA 08	IP2H&S INX		
AB 8D 20	P2H&S BSR	P2HEX	
AD 86 20	PNTSP LDA A	#320	(space)
AF 20 CE	BRA	PNTCH	(BR & RTS)

Get address routine; accepts four hex digits from keyboard, converts them to a two byte address stored in TMPXHI, TMPXLO and also in X.

FFB1 8D FA	GETADR BSR	PNTSP	
B3 8D 0D	BSR	HEX2IN	
B5 B7 FOF6	STA A	TMPXHI	
B8 8D 08	BSR	HEX2IN	
BA B7 FOF7	STA A	TMPXLO	
BD FE FOF6	LDX	TMPXHI	copy addr into X
CO 20 EB	BRA	PNTSP	(BR & RTS)

HEX2IN routine gets two characters from keyboard, treats them as hex digits ('0'-'9', 'A'-'F') then converts them to one binary byte in A. Exits to RSTART if a non-hex character inputs.

FFC2 8D 1F	HEX2IN BSR	GETHEX	get 1st. hex dig
C4 48	ASL A		
C5 48	ASL A		shift it left 4
C6 48	ASL A		bits
C7 48	ASL A		
C8 16	TAB		save it
C9 8D 18	BSR	GETHEX	get 2nd. hex dig
CB 1B	ABA		& combine them
CC 39	RTS		

P2HEX prints byte pointed to by X as 2 hex char.

FFCD A6 00	P2HEX LDA A	0,X	get byte
CF 47	ASR A		
DO 47	ASR A		shift it right
D1 47	ASR A		4 bits
D2 47	ASR A		
D3 8D 02	BSR	PNTHEX	print 1st. hex
D5 A6 00	LDA A	0,X	retrieve byte
D7 84 0F	PNTHEX AND A	#80F	blank left 4 bits
D9 8B 30	ADD A	#830	convert to ASCII
DB 81 39	CMP A	#839	
DD 23 AO	BLS	PNTCH	(BR & RTS)
DF 8B 07	ADD A	#87	
E1 20 9C	BRA	PNTCH	

GETHEX takes an ASCII character from the keyboard and converts it to Hex if it is in the range '0'-'9', 'A'-'F'. Else it jumps to RSTART

FFE3	8D 96	GETHEX	BSR	GETCH	get ASCII char
E5	80 30		SUB	A # 8 30	
E7	2B 90		BMI	START2	
E9	81 09		CMP	A # 8 9	
EB	2F 08		BLE	RTS2	
ED	80 07		SUB	A # 8 7	
EF	2B 88		BMI	START2	
F1	81 0F		CMP	A # 8 F	
F3	2E 81		BGT	START2	
F5	39	RTS2	RTS		

FFF6 & FFF7; space for expansion

Interrupt vectors;

FFF8	FF1B	SWINT	(IRQ vector)
FFFA	FF1B	SWINT	(SWI ")
FFFC	FF00	NMIHDL	(NMI ")
FFFE	FF0B	RESET	(RESET ")

7768 VDU 1

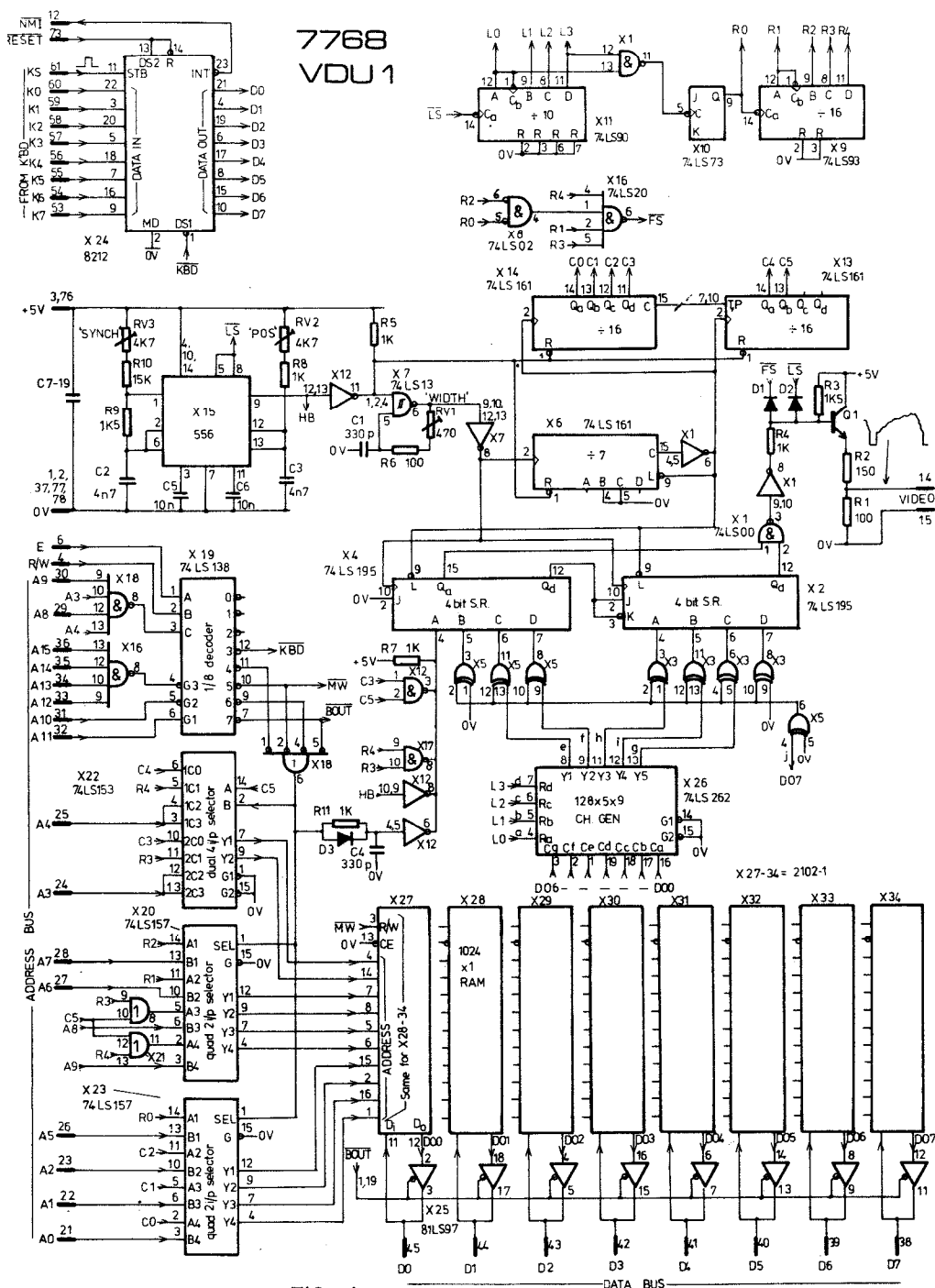
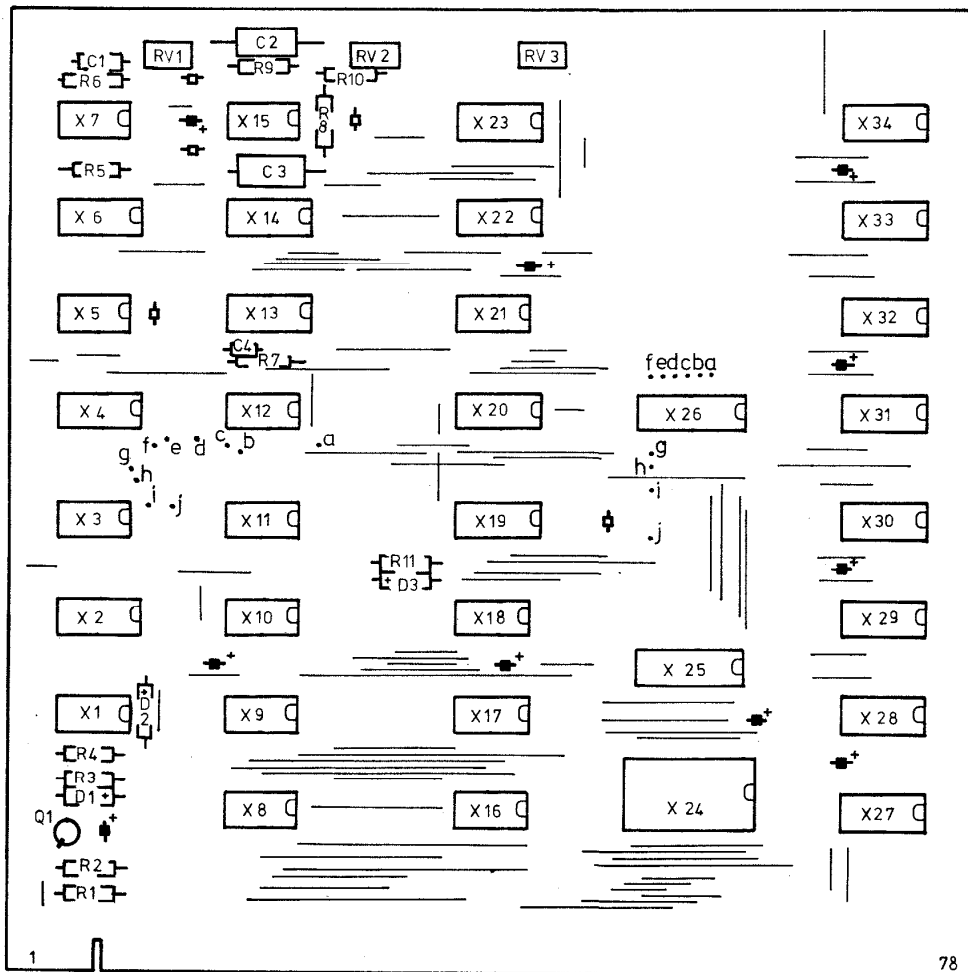


FIG. 1

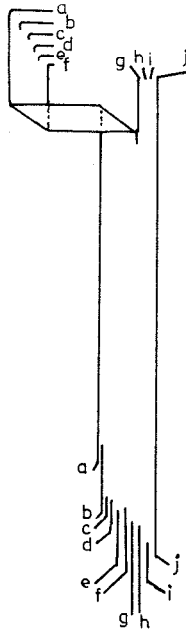


 C10-19
 C5-9

Ribbon cable (see Fig 3) connects points a,b,c,d ... j

77-68 VDU BOARD COMPONENT SIDE VIEW

Fig 2



not to scale.

use miniature 10 way solid core cable e.g. Doram 357-491

77-68 VDU 1 BOARD

Ribbon Cable

Fig 3

77-68 VDU 1 BOARD

Components

Logic

X1	74LS 00 ; quad 2 i/p nand	X15	556 ; dual timer
X2,4	" 195 ; 4 stage shift reg	X16,18	74LS 20 ; dual 4 i/p nand
X3,5	" 86 ; quad exclusive or	X19	" 138 ; 1/8 decoder
X6,13,14	" 161 ; 4 stage counter	X20,23	" 157 ; quad 2 i/p sel
X7	" 13 ; dual schmidt	X21	" 32 ; quad 2 i/p or
X8	" 02 ; quad 2 i/p nor	X22	" 153 ; dual 4i/p sel
X9	" 93 ; divide by 16	X24	8212 ; 8 bit i/p port
X10	" 73 ; dual J-K	X25	81LS97 ; 8 bit buffer
X11	" 90 ; divide by 10	X26	74S262 ; char gen
X12,17	" 03 ; quad o/c nand	X27-34	2102-1 ; 1024 x 1 RAM

Notes; X27-34 must be 450ns or better access time.

X24 may also be 74S412.

Transistors

Q1 2N2222

Diodes

D1,2,3 1N4148

Resistors

All are miniature types.

R1,6	100 ohm	RV1	100 ohm vertical preset (min)
R2	150 "	RV2,3	4k7 ohm " " "
R3,9	1k5 "		
R4,5,7,8,11	1k0 "		
R10	15k "		

Capacitors

C1,4	330 pF polystyrene
C2,3	4700 pF polystyrene
C5-9	10 nF ceramic
C10-19	6.8 uF 10V tantalum bead

Misc

DIL sockets ; if fitted then ; 1 off 24 pin
2 off 20 pin
16 off 16 pin
15 off 14 pin

77-68 VDU 1 printed circuit board.

Ribbon cable - miniature solid cored type.

Edge connector - 77 way + 1 polarising position (78 positions total).

F800	F807	F808	F80F	F810	F817	F818	F81F	FB00	FB07
F820	F827	F828	F82F	F830	F837	F838	F83F	FB20	FB27
F840	F847	F848	F84F	F850	F857	F858	F85F	FB40	FB47
F860	F867	F868	F86F	F870	F877	F878	F87F	FB60	FB67
F880	F887	F888	F88F	F890	F897	F898	F89F	FB80	FB87
F8A0	F8A7	F8A8	F8AF	F8B0	F8B7	F8B8	F8BF	FBA0	FBA7
F8C0	F8C7	F8C8	F8CF	F8D0	F8D7	F8D8	F8DF	FBC0	FBC7
F8E0	F8E7	F8E8	F8EF	F8F0	F8F7	F8F8	F8FF	FBE0	FBE7
F900	F907	F908	F90F	F910	F917	F918	F91F	FB08	FB0F
F920	F927	F928	F92F	F930	F937	F938	F93F	FB28	FB2F
F940	F947	F948	F94F	F950	F957	F958	F95F	FB48	FB4F
F960	F967	F968	F96F	F970	F977	F978	F97F	FB68	FB6F
F980	F987	F988	F98F	F990	F997	F998	F99F	FB88	FB8F
F9A0	F9A7	F9A8	F9AF	F9B0	F9B7	F9F8	F9BF	FBA8	FBAF
F9C0	F9C7	F9C8	F9CF	F9D0	F9D7	F9D8	F9DF	FBC8	FBCF
F9E0	F9E7	F9E8	F9EF	F9F0	F9F7	F9F8	F9FF	FBE8	FBEF
FA00	FA07	FA08	FA0F	FA10	FA17	FA18	FA1F	FB10	FB17
FA20	FA27	FA28	FA2F	FA30	FA37	FA38	FA3F	FB30	FB37
FA40	FA47	FA48	FA4F	FA50	FA57	FA58	FA5F	FB50	FB57
FA60	FA67	FA68	FA6F	FA70	FA77	FA78	FA7F	FB70	FB77
FA80	FA87	FA88	FA8F	FA90	FA97	FA98	FA9F	FB90	FB97
FAA0	FAA7	FAA8	FAAF	FAB0	FAB7	FAB8	FABF	FBA0	FBA7
FAC0	FAC7	FAC8	FACF	FAD0	FAD7	FAD8	FADF	FBC0	FBC7
FAE0	FAE7	FAE8	FAEF	FAF0	FAF7	FAF8	FAFF	FBE0	FBE7

← 8 char →

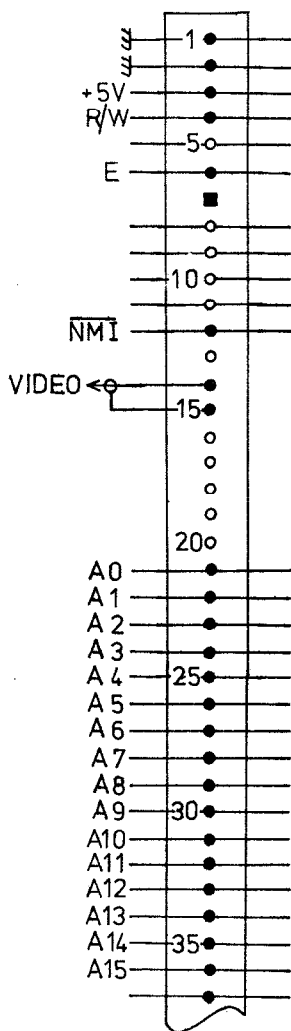
← 40 char →

Keyboard buffer addr = FBFF

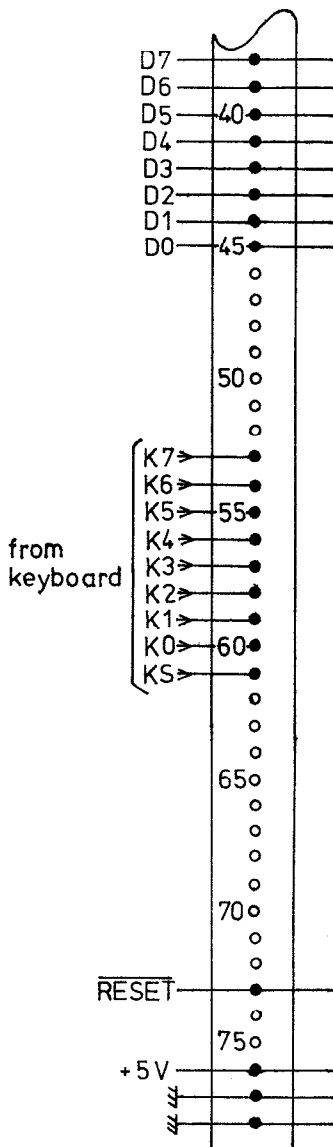
MEMORY MAP

Memory addresses for VDU screen locations

Fig 5



- Polarising key
- Bus line used by VDU
- " " not used
- Other connection to board

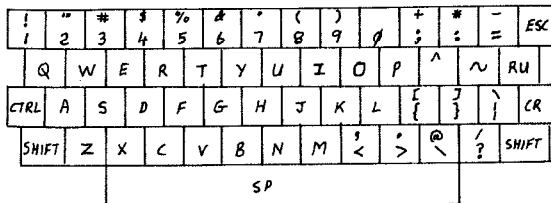
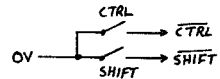
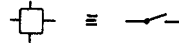
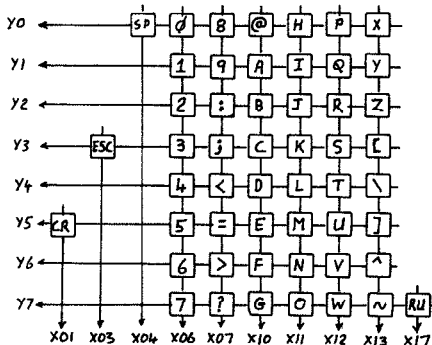
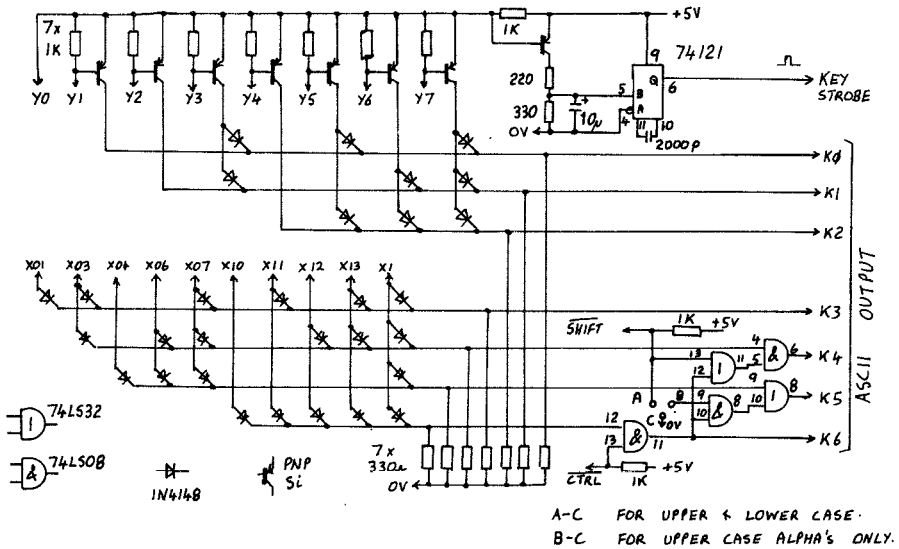


REAR VIEW

77-68 VDU1 BOARD

CONNECTOR WIRING

Fig 6



SIMPLE ASCII KEYBOARD

Fig 7